Documentation d'installation de Gatus sous Debian 12

Sommaire :

1. Objectif 3
2. Installation de Golang 3
Étape 1 : Mettre à jour le système 3
Étape 2 : Installer Golang depuis les dépôts Debian 3
Étape 3 : Désinstaller l'ancienne version de Go 3
Étape 4 : Télécharger la dernière version officielle de Go 3
Étape 5 : Supprimer l'ancienne installation de Go 3
Étape 6 : Extraire et installer la nouvelle version 4
Étape 7 : Ajouter Go au PATH du système 4
Étape 8 : Vérifier la version de Go installée 4
3. Téléchargement et compilation de Gatus 4
Étape 1 : Installer Git 4
Étape 2 : Cloner le dépôt Git de Gatus 5
Étape 3 : Se placer dans le dossier Gatus 5
Étape 4 : Compiler Gatus 5
4. Test initial avec un seul service 5
Étape 1 : Créer un fichier de configuration minimal 5
Étape 2 : Lancer Gatus avec ce fichier 5
Étape 3 : Vérifier dans le navigateur 6
5. Configuration complète de Gatus 6
6. Création du service systemd 7
Objectif : 7
Étape 1 : Créer le fichier de service 8
Étape 2 : Activer et démarrer le service 8
7. Accès à l'interface web 8
8. Test du système d'alerte 9
Étape 1 : Modifier un site pour le rendre inaccessible 9
Étape 2 : Redémarrer Gatus 9
9. Conclusion 10

1. Objectif

Cette documentation décrit l'installation, la configuration et la mise en service de Gatus sur Debian 12. Gatus est un outil open-source permettant de superviser des sites web, des API et des services réseau. Il offre une interface web ainsi que l'envoi d'alertes automatiques en cas de défaillance.

2. Installation de Golang

Gatus est développé en langage Go. Avant de le compiler, il est nécessaire d'installer et de configurer l'environnement Golang.

Étape 1 : Mettre à jour le système

Nous commençons par mettre à jour la liste des paquets et les versions installées du système Debian.

```
sudo apt update && sudo apt upgrade -y
```

Cette commande :
met à jour la base de données des paquets (apt update),
installe toutes les mises à jour disponibles (apt upgrade -y).

Étape 2 : Installer Golang depuis les dépôts Debian

```
sudo apt install golang-go -y
```

Cette commande installe la version de Go disponible dans les dépôts officiels Debian.
Cependant, cette version peut être trop ancienne pour compiler les versions récentes de Gatus.

Étape 3 : Désinstaller l'ancienne version de Go

Avant d'installer la dernière version, on supprime celle installée depuis les dépôts.

```
sudo apt remove golang-go -y
```

Cela permet d'éviter les conflits entre les deux installations.

Étape 4 : Télécharger la dernière version officielle de Go

On se place dans le répertoire /tmp pour télécharger l'archive depuis le site officiel de Go :

```
cd /tmp
wget https://go.dev/dl/go1.24.4.linux-amd64.tar.gz
```

La commande wget télécharge le fichier compressé contenant l'environnement Go version 1.24.4.

Étape 5 : Supprimer l'ancienne installation de Go

```
sudo rm -rf /usr/local/go
```

Cette commande efface complètement le dossier contenant une ancienne version de Go

Étape 6 : Extraire et installer la nouvelle version

```
sudo tar -C /usr/local -xzf go1.24.4.linux-amd64.tar.gz
```

Cette commande extrait le contenu de l'archive téléchargée et installe Go dans /usr/local/go.

Étape 7 : Ajouter Go au PATH du système

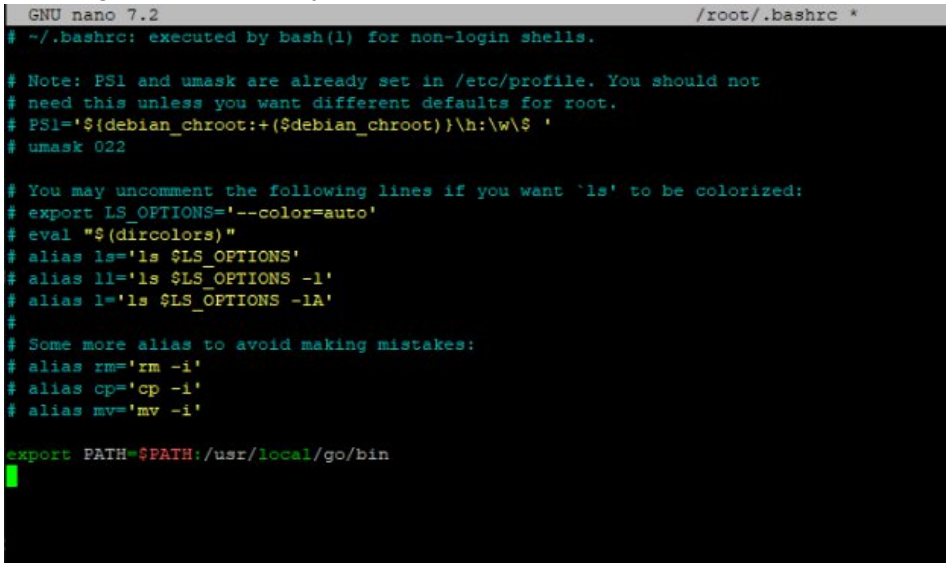
Ouvrir le fichier .bashrc pour y ajouter le chemin d'accès à Go :

```
nano ~/.bashrc
```

Ajouter à la fin du fichier :

```
export PATH=$PATH:/usr/local/go/bin
```

Cette ligne indique au système où trouver les exécutables de Go.



Recharger le fichier afin que la modification prenne effet immédiatement :

```
source ~/.bashrc
```

Étape 8 : Vérifier la version de Go installée

```
go version
```

Si l'installation est correcte, le terminal doit afficher :

```
go version go1.24.7 linux/amd64
```

3. Téléchargement et compilation de Gatus

Étape 1 : Installer Git

Gatus est disponible sur GitHub.
On installe donc Git pour pouvoir cloner le dépôt :

```
sudo apt install git -y
```

Étape 2 : Cloner le dépôt Git de Gatus

```
git clone https://github.com/TwiN/gatus.git
```

Cette commande télécharge le code source complet de Gatus dans un dossier appelé gatus.

Étape 3 : Se placer dans le dossier Gatus

```
cd gatus
```

Cela permet de se positionner dans le répertoire contenant le code source.

Étape 4 : Compiler Gatus

```
go build -o gatus .
```

Cette commande demande à Go de compiler le code source et de créer un exécutable nommé gatus

4. Test initial avec un seul service

Avant d'aller plus loin, on va effectuer un premier test de Gatus avec un service simple : **Google**.

Étape 1 : Créer un fichier de configuration minimal

```
nano config.yaml
```

Insérer le contenu suivant :

```
endpoints:
  - name: Google
    url: "https://www.google.com"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 500"
./gatus serve --config=config.yaml
```

```
GNU nano 7.2
endpoints:
  - name: Google
    url: "https://www.google.com"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 500"
```

Ce fichier indique à Gatus de vérifier le site Google toutes les minutes, et de considérer le service comme fonctionnel si le code HTTP est 200 et le temps de réponse inférieur à 500 ms.

Étape 2 : Lancer Gatus avec ce fichier

```
./gatus serve --config=config.yaml
```

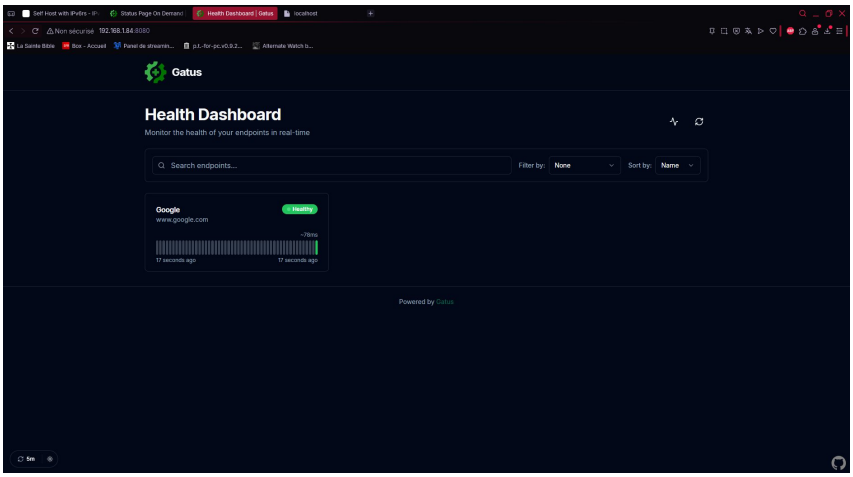
```
Fiber v2.52.9
http://[::]:8080

Handlers ........... 39  Processes ........... 1
Prefork ....... Disabled  PID ............... 5934
```

Gatus démarre et affiche un message indiquant qu'il écoute sur le port 8080.

Étape 3 : Vérifier dans le navigateur

Accéder à l'adresse suivante :

```
http://192.168.1.84:8080
```



Gatus affiche un tableau de bord avec le service Google en vert (Healthy).

5. Configuration complète de Gatus

Après ce test, on crée une configuration plus avancée, avec plusieurs services, stockage en base SQLite et alertes Discord.

```
nano /root/gatus/config/config.yaml
```

Voici un exemple de ce que l'on peut faire :

```yaml
# ============================================================
# GATUS - Configuration
# ============================================================

# --------------------------
# STOCKAGE DES DONNÉES
# --------------------------
storage:
  type: sqlite
  path: data.db

# --------------------------
# ALERTING (DISCORD)
# --------------------------
alerting:
  discord:
    webhook-url: "https://discord.com/api/webhooks/1431034467715121274/ujVeMdKwHsg0RcGhYrGixhq9RpWQnHFjQj
    default-alert:
      description: "Un service surveillé par Gatus est en panne 🚨"
      send-on-resolved: true

# --------------------------
# ENDPOINTS - SERVICES À SURVEILLER
# --------------------------
endpoints:
  # ---- Production ----
  - name: Mon Site Web Principal
    group: Production
    url: "https://kalandy.carrd.co"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 500"
      - "[CERTIFICATE_EXPIRATION] > 72h"
    alerts:
      - type: discord
        failure-threshold: 3
        success-threshold: 2

  # ---- Local Network ----
  - name: Serveur Web Local
    group: Réseau Local
    url: "http://192.168.1.84:8080/health"
    interval: 30s
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 300"

  - name: NAS - ICMP Ping
    group: Réseau Local
    url: "icmp://192.168.1.20"
    interval: 1m
    conditions:
      - "[CONNECTED] == true"

  # ---- DNS Check ----
  - name: DNS Google
    group: Réseau
    url: "8.8.8.8"
    interval: 1m
    dns:
      query-name: "google.com"
      query-type: "A"
    conditions:
      - "[BODY] == pat(*.*.*.*)"
      - "[DNS_RCODE] == NOERROR"

  # ---- Test externe ----
  - name: API Github
    group: Externe
    url: "https://api.github.com"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 400"
```

6. Création du service systemd

Objectif :

Créer un service permet à Gatus :

- de démarrer automatiquement au lancement du système,
- de s’exécuter en arrière-plan,

- de redémarrer automatiquement en cas de crash.

Étape 1 : Créer le fichier de service

```
sudo nano /etc/systemd/system/gatus.service
```

Contenu du fichier :

```
[Unit]
Description=Gatus monitoring service
After=network.target

[Service]
User=root
WorkingDirectory=/root/gatus
ExecStart=/root/gatus/gatus serve
Restart=always

[Install]
WantedBy=multi-user.target
```

```
[Unit]
Description=Gatus monitoring service
After=network.target

[Service]
User=root
WorkingDirectory=/root/gatus
ExecStart=/root/gatus/gatus serve
Restart=always

[Install]
WantedBy=multi-user.target
```

Étape 2 : Activer et démarrer le service

```
sudo systemctl daemon-reload
sudo systemctl enable gatus
sudo systemctl start gatus
sudo systemctl status gatus
```

```
● gatus.service - Gatus monitoring service
     Loaded: loaded (/etc/systemd/system/gatus.service; enabled; preset: enabled)
     Active: active (running) since Fri 2025-10-24 00:15:10 CEST; 20min ago
   Main PID: 6216 (gatus)
      Tasks: 6 (limit: 2315)
     Memory: 16.8M
        CPU: 662ms
     CGroup: /system.slice/gatus.service
             └─6216 /root/gatus/gatus serve
```

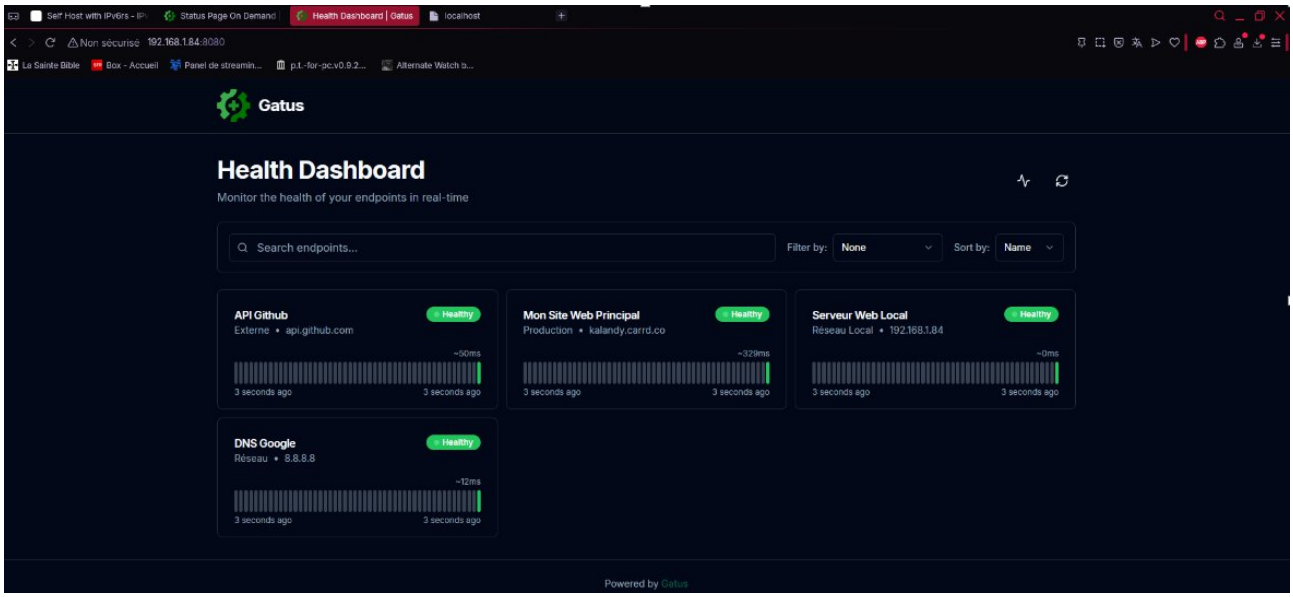
Le service doit être actif et afficher l'état "running".

7. Accès à l'interface web

Une fois le service démarré, accéder à :

```
http://192.168.1.84:8080
```

Le tableau de bord affiche tous les services surveillés, leur état et leur temps de réponse.



8. Test du système d'alerte

Afin de créer une alerte, nous allons volontairement provoquer une erreur.

Étape 1 : Modifier un site pour le rendre inaccessible

Ouvrir la configuration :

```
nano config.yaml
```

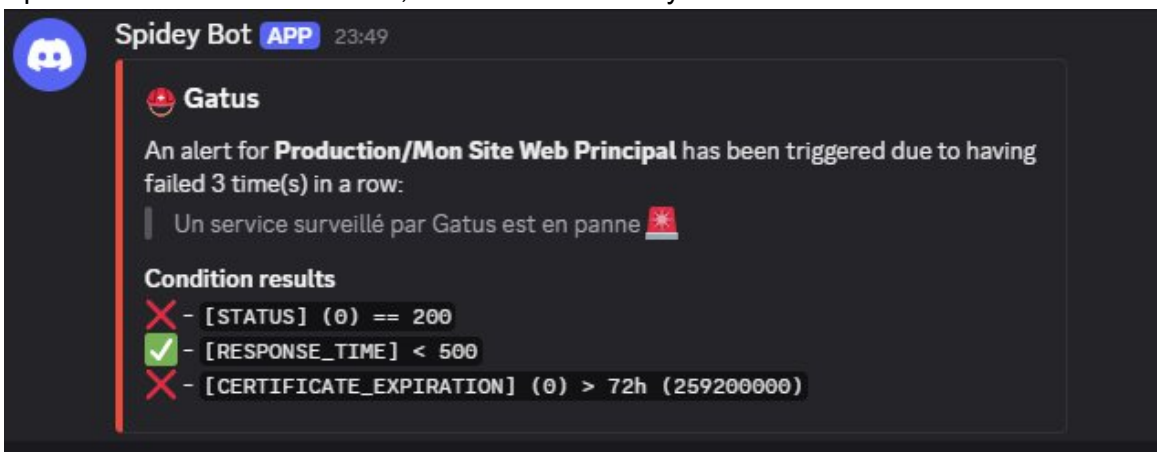
Modifier l'URL d'un service par un site inexistant :

```
url: "https://fake.kalandy.carrd.co"
```

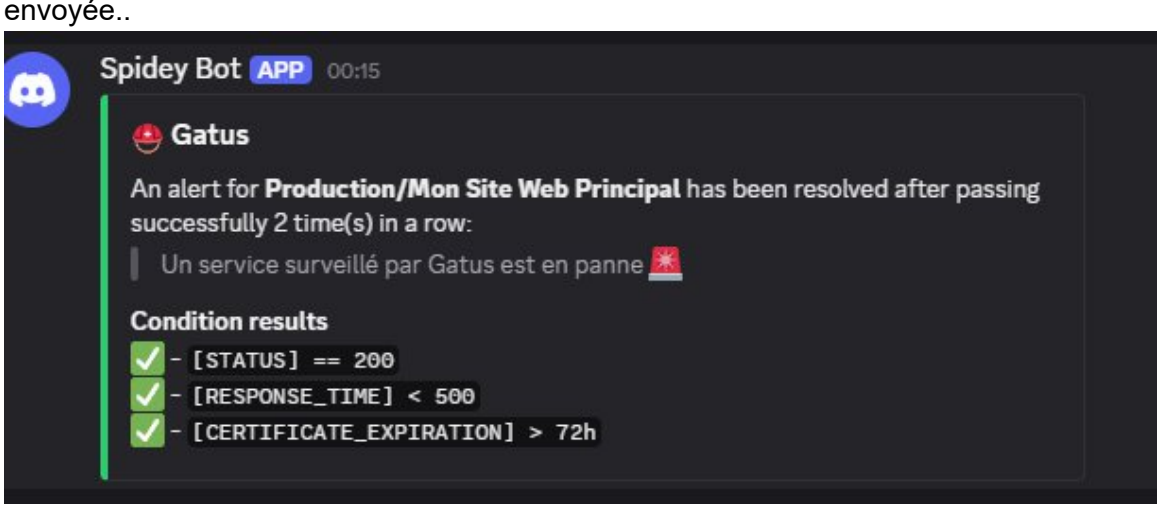
Étape 2 : Redémarrer Gatus

```
sudo systemctl restart gatus
```

Après trois échecs consécutifs, une alerte est envoyée via Discord.



Lorsque le service redevient accessible, une notification de rétablissement est automatiquement envoyée..



9. Conclusion

Gatus est désormais entièrement configuré.
Il permet de :

- superviser plusieurs services internes et externes,
- conserver l'historique des vérifications,
- envoyer des alertes via Discord,
- et redémarrer automatiquement grâce à systemd.

Ce système fournit une solution simple, moderne et fiable de supervision réseau sur Debian 12.